ECE2002
DIGITAL LOGIC DESIGN

Module 1 – Lecture 1

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Positional Number Systems

Decimal Numbers: A decimal number (base 10 or radix 10) such as 7392 can be represented as:

$$7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0$$

In general, a number with a decimal point is represented by a series of coefficients:

$$a_5\, a_4\, a_3\, a_2\, a_1\, a_0 \,.\, a_{-1}\, a_{-2}\, a_{-3}$$

The coefficients a_j are any of the 10 digits (0, 1, 2, …, 9), and the subscript value j gives the place value and, hence, the power of 10 by which the coefficient must be multiplied. Thus, the preceding decimal number can be expressed as

$$10^5 a_5 + 10^4 a_4 + 10^3 a_3 + 10^2 a_2 + 10^1 a_1 + 10^0 a_0 + 10^{-1} a_{-1} + 10^{-2} a_{-2} + 10^{-3} a_{-3}$$

with $a_3 = 7$, $a_2 = 3$, a $a_1 = 9$, and $a_0 = 2$.

In general, a number expressed in a base-r system has coefficients multiplied by powers of r:

$$a_n \,.\, r^n + a_{n-1} \,.\, r^{n-1} + \ldots + a_2 \,.\, r^2 + a_1 \,.\, r + a_0 + a_{-1} \,.\, r^{-1} + a_{-2} \,.\, r^{-2} + \ldots + a_{-m} \,.\, r^{-m}$$

The coefficients a_j range in value from 0 to r - 1.

Dr. Arindam Dutta

Positional Number Systems

Binary Numbers: The coefficients of the binary number system have only two possible values: 0 and 1.

Each coefficient a_j is multiplied by a power of the radix, e.g., 2^j, and the results are added to obtain the decimal equivalent of the number.

For example, the decimal equivalent of the binary number 11010.11 is 26.75, as shown from the multiplication of the coefficients by powers of 2:

$$1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 26.75$$

Octal Numbers: The octal number system is a base-8 system that has eight digits: 0, 1, 2, 3, 4, 5, 6, 7. An example of an octal number is 127.4. To determine its equivalent decimal value, we expand the number in a power series with a base of 8:

$$(127.4)_8 = 1 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1} = (87.5)_{10}$$

Dr. Arindam Dutta

Positional Number Systems

Hexadecimal Numbers: Here, the first 10 digits are borrowed from the decimal system. The letters A, B, C, D, E, and F are used for the digits 10, 11, 12, 13, 14, and 15, respectively.

An example of a hexadecimal number is

$$(B65F)_{16} = 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0 = (46687)_{10}$$

The hexadecimal system is used commonly by designers to represent long strings of bits in the addresses, instructions, and data in digital systems.

For example, B65F is used to represent 1011011001010000.

Dr. Arindam Dutta

Positional Number Systems

Number-Base Conversion

- Representations of a number in a different radix are said to be equivalent if they have the same decimal representation. For example, $(0011)_8$ and $(1001)_2$ are equivalent—both have decimal value 9.
- The conversion of a number in base *r* to decimal is done by expanding the number in a power series and adding all the terms.
- The conversion of a decimal integer to a number in base *r* is done by dividing the number and all successive quotients by *r* and accumulating the remainders.
- In the next slides we will see the reverse operation of converting a decimal number to a number in base *r* and the conversion between other bases.
- If the number includes a radix point, it is necessary to separate the number into an integer part and a fraction part, since each part must be converted differently.
- Number consisting of both integer and fraction parts are called **fixed-point numbers**.

Dr. Arindam Dutta

Positional Number Systems

Example: Convert decimal 41 to binary.

First, 41 is divided by 2 to give an integer quotient of 20 and a remainder of 12. Then the quotient is again divided by 2 to give a new quotient and remainder. The process is continued until the integer quotient becomes 0. The coefficients of the desired binary number are obtained from the remainders as follows:

	Integer Quotient		Remainder	Coefficient
$41/2 =$	20	+	1	$a_0 = 1$
$20/2 =$	10	+	0	$a_1 = 0$
$10/2 =$	5	+	0	$a_2 = 0$
$5/2 =$	2	+	1	$a_3 = 1$
$2/2 =$	1	+	0	$a_4 = 0$
$1/2 =$	0	+	1	$a_5 = 1$



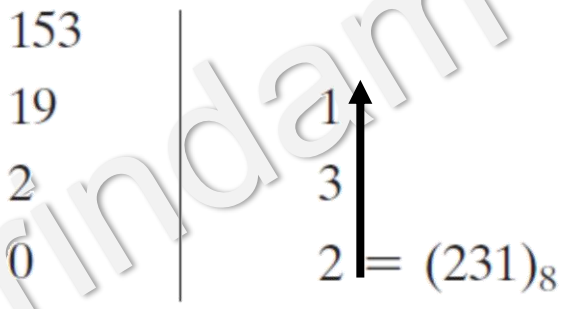
Therefore, the answer is $(41)_{10} = (a_5a_4a_3a_2a_1a_0)_2 = (101001)_2$.

Dr. Arindam Dutta

Positional Number Systems

Example: Convert decimal 153 to octal.

The required base r is 8. First, 153 is divided by 8 to give an integer quotient of 19 and a remainder of 1. Then 19 is divided by 8 to give an integer quotient of 2 and a remainder of 3. Finally, 2 is divided by 8 to give a quotient of 0 and a remainder of 2. This process can be conveniently manipulated as follows:



Dr. Arindam Dutta

Positional Number Systems

Example: Convert $(0.6875)_{10}$ to binary.

First, 0.6875 is multiplied by 2 to give an integer and a fraction. Then the new fraction is multiplied by 2 to give a new integer and a new fraction. The process is continued until the fraction becomes 0 or until the number of digits has sufficient accuracy. The coefficients of the binary number are obtained from the integers as follows:

	Integer		Fraction	Coefficient
$0.6875 \times 2 =$	1	+	0.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0	+	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1	+	0.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1	+	0.0000	$a_{-4} = 1$

Therefore, the answer is $(0.6875)_{10} = (0.\ a_{-1}\ a_{-2}\ a_{-3}\ a_{-4})_2 = (0.1011)_2$.

Dr. Arindam Dutta

Positional Number Systems

Example: Convert $(0.513)_{10}$ to octal.

$$
\begin{array}{lcll}
0.513 & \times & 8 = & 4.104 \quad 4 \\
0.104 & \times & 8 = & 0.832 \quad 0 \\
0.832 & \times & 8 = & 6.656 \quad 6 \\
0.656 & \times & 8 = & 5.248 \quad 5 \\
0.248 & \times & 8 = & 1.984 \quad 1 \\
0.984 & \times & 8 = & 7.872 \quad 7
\end{array}
\Bigg\downarrow
$$

The answer, to seven significant figures, is obtained from the integer part of the products: $(0.513)_{10} = (0.406517)_8$

The conversion of decimal numbers with both integer and fraction parts is done by converting the integer and the fraction separately and then combining the two answers.

Using the results of sides 6 and 8, we obtain $(41.6875)_{10} = (101001.1011)_2$

From slides 7 and 9, we have $(153.513)_{10} = (231.406517)_8$

Dr. Arindam Dutta

Positional Number Systems

Numbers with different bases

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Dr. Arindam Dutta

Positional Number Systems

Example: Convert $(101100011010 11.111000001 10)_2$ to octal.

The conversion from binary to octal is easily accomplished by partitioning the binary number into groups of three digits each, starting from the binary point and proceeding to the left and to the right. The corresponding octal digit is then assigned to each group.

$$\begin{array}{cccccccccc} (10 & 110 & 001 & 101 & 011 & \cdot & 111 & 100 & 000 & 110)_2 = (26153.7406)_8 \\ 2 & 6 & 1 & 5 & 3 & & 7 & 4 & 0 & 6 \end{array}$$

Example: Convert $(101100011010 11.1110010)_2$ to hexadecimal.

Conversion from binary to hexadecimal is similar, except that the binary number is divided into groups of four digits:

$$\begin{array}{cccccccc} (10 & 1100 & 0110 & 1011 & \cdot & 1111 & 0010)_2 = (2C6B.F2)_{16} \\ 2 & C & 6 & B & & F & 2 \end{array}$$

Dr. Arindam Dutta

Positional Number Systems

Example: **Convert $(673.124)_8$ to binary.**

Conversion from octal to binary is done by converting each octal digit to its three-digit binary equivalent:

$$\begin{array}{lccccccc} (673.124)_8 = (110 & 111 & 011 & \cdot & 001 & 010 & 100)_2 \\ \quad\;\; 6 & 7 & 3 & & 1 & 2 & 4 \end{array}$$

Example: **Convert $(306.D)_{16}$ to binary.**

Conversion from hexadecimal to binary is done by converting each hexadecimal digit to its four-digit binary equivalent:

$$\begin{array}{lcccccc} (306.D)_{16} = (0011 & 0000 & 0110 & \cdot & 1101)_2 \\ \quad\;\; 3 & 0 & 6 & & D \end{array}$$

Dr. Arindam Dutta

Positional Number Systems

Binary addition: The binary addition is performed by using the following rules:

(i) 0 + 0 = 0
(ii) 1 + 0 = 1
(iii) 0 + 1 = 1
(iv) 1 + 1 = 10 (since, decimal equivalent of 10 is 2)

Example:

$$\begin{array}{r} 10001 (= 17) \\ +111 (= 7) \\ \hline 11000 (= 24) \end{array}$$

- First column 1 + 1 = 10, place 0 and carry 1
- Second column, 1 + 0 = 1, 1 + 1 = 10, place 0 and carry 1
- Third column, 1 + 0 = 1, 1 + 1 = 10, place 0 and carry 1
- Fourth column, 1 + 0 = 1, place 1 and carry 0
- Fifth column, 1 + 0 = 1, place 1

Dr. Arindam Dutta

Positional Number Systems

Binary subtraction: The binary subtraction is performed by using the following rules:

(i) 0 – 0 = 0
(ii) 1 – 0 = 1
(iii) 1 – 1 = 0
(iv) 10 – 1 = 1 (decimal equivalent of 10 is binary 2)

Example:

$$\begin{array}{r} 110\,(=6) \\ -11\,(=3) \\ \hline 011\,(=3) \end{array}$$

- The first column is 10 – 1 = 1 (after borrowing 1 from the second column).
- The borrow is carried and added to the lower number of second column to get 1 + 1 = 10.
- So, the second column gives 1 – 0 = 1 and the third column gives 1 – 1 = 0.

Dr. Arindam Dutta

Positional Number Systems

Binary multiplication: Binary multiplication is carried out using the following rules:

(i) $0 \times 0 = 0$
(ii) $1 \times 0 = 0$
(iii) $0 \times 1 = 0$
(iv) $1 \times 1 = 1$

Example:

		1	1	1	(= 7)	
		1	0	1	(= 5)	
		1	1	1		
	0	0	0	×		
1	1	1	×	×		
1	0	0	0	1	1	(= 35)
1 + 1 = 10		1 + 1 = 10	1 + 1 = 10	1 + 0 = 1	1 + 0 = 1	
		carry 1	carry 1			

Dr Arindam Dutta

Positional Number Systems

Complements of Numbers

- Used in digital computers to simplify the subtraction operation and for logical manipulation.
- There are two types of complements for each base-*r* system:
  - the radix complement, and
  - the diminished radix complement.
- The first is referred to as the *r*'s complement and the second as the (*r* - 1)'s complement.
- When the value of the base *r* is substituted in the name, the two types are referred to as the 2's complement and 1's complement for binary numbers and the 10's complement and 9's complement for decimal numbers.

Dr. Arindam Dutta

Positional Number Systems

Diminished Radix Complement

Given a number N in base r having n digits, the $(r - 1)$'s complement of N, i.e., its diminished radix complement, is defined as $(r^n - 1) - N$.

For decimal numbers, $r = 10$ and $r - 1 = 9$, so the 9's complement of N is $(10^n - 1) - N$.

In this case, 10^n represents a number that consists of a single 1 followed by n 0's. $10^n - 1$ is a number represented by n 9's.

For example, if $n = 4$, we have $10^4 = 10{,}000$ and $10^4 - 1 = 9999$. It follows that the 9's complement of a decimal number is obtained by subtracting each digit from 9. Examples:

The 9's complement of 546700 is 999999 - 546700 = 453299.

The 9's complement of 012398 is 999999 - 012398 = 987601.

Dr. Arindam Dutta

Positional Number Systems

For binary numbers, $r = 2$ and $r - 1 = 1$, so the 1's complement of N is $(2^n - 1) - N$. Again, 2^n is represented by a binary number that consists of a 1 followed by n 0's. $2^n - 1$ is a binary number represented by n 1's.

For example, if $n = 4$, we have $2^4 = (10000)_2$ and $2^4 - 1 = (1111)_2$. Thus, the 1's complement of a binary number is obtained by subtracting each digit from 1.

However, when subtracting binary digits from 1, we can have either $1 - 0 = 1$ or $1 - 1 = 0$, which causes the bit to change from 0 to 1 or from 1 to 0, respectively.

Therefore, **the 1's complement of a binary number is formed by changing 1's to 0's and 0's to 1's.** Examples:

The 1's complement of 1011000 is 0100111.

The 1's complement of 0101101 is 1010010.

The $(r - 1)$'s complement of octal or hexadecimal numbers is obtained by subtracting each digit from 7 or F (decimal 15), respectively.

Dr. Arindam Dutta

Positional Number Systems

Radix Complement: The r's complement of an n-digit number N in base r is defined as $r^n - N$ for $N \neq 0$ and as 0 for $N = 0$. Comparing with the $(r - 1)$'s complement, we note that the r's complement is obtained by adding 1 to the $(r - 1)$'s complement, since $r^n - N = [(r^n - 1) - N] + 1$.

Thus, the 10's complement of decimal 2389 is 7610 + 1 = 7611 and is obtained by adding 1 to the 9's complement value.

The 2's complement of binary 101100 is 010011 + 1 = 010100 and is obtained by adding 1 to the 1's-complement value.

Since 10^n is a number represented by a 1 followed by n 0's, $10^n - N$, which is the 10's complement of N, can be formed also by leaving all least significant 0's unchanged, subtracting the first nonzero least significant digit from 10, and subtracting all higher significant digits from 9. Thus,

10's complement of 012398 is 987602, and

10's complement of 246700 is 753300

Dr. Arindam Dutta

Positional Number Systems

The 10's complement of the first number is obtained by subtracting 8 from 10 in the least significant position and subtracting all other digits from 9. The 10's complement of the second number is obtained by leaving the two least significant 0's unchanged, subtracting 7 from 10, and subtracting the other three digits from 9.

Similarly, the **2's complement can be formed by leaving all least significant 0's and the first 1 unchanged and replacing 1's with 0's and 0's with 1's in all other higher significant digits**. For example,

2's complement of 1101100 is 0010100, and

2's complement of 0110111 is 1001001

The 2's complement of the first number is obtained by leaving the two least significant 0's and the first 1 unchanged and then replacing 1's with 0's and 0's with 1's in the other four most significant digits.

The 2's complement of the second number is obtained by leaving the least significant 1 unchanged and complementing all other digits.

Note: *The nonzero digit of a number that is the farthest left is the most significant digit, and the nonzero digit farthest right is the least significant digit.*

Dr. Arindam Dutta

Positional Number Systems

Subtraction with Complements

A 1 is borrowed from a higher significant position when the minuend digit is smaller than the subtrahend digit.

The subtraction of two n-digit unsigned numbers M - N in base r can be done as follows:

1. Add the minuend M to the r's complement of the subtrahend N. Mathematically,

$$\boldsymbol{M + (r^n - N) = M - N + r^n}$$

2. If $M \geq N$, the sum will produce an end carry r^n, which can be discarded; what is left is the result M - N.

3. If $M < N$, the sum does not produce an end carry and is equal to r^n - (N - M), which is the r's complement of (N - M). To obtain the answer in a familiar form, take the r's complement of the sum and place a negative sign in front.

Dr. Arindam Dutta

Positional Number Systems

Example: Using 10's complement, subtract 72532 - 3250.

$$
\begin{aligned}
M &= \quad 72532 \\
\text{10's complement of } N &= + \quad 96750 \\
\text{Sum} &= \quad 169282 \\
\text{Discard end carry } 10^5 &= - 100000 \\
\textit{Answer} &= \quad 69282
\end{aligned}
$$

Note that M has five digits and N has only four digits. Both numbers must have the same number of digits, so we write N as 03250.

Taking the 10's complement of N produces a 9 in the most significant position.

The occurrence of the end carry signifies that $M \geq N$ and that the result is therefore positive.

Dr. Arindam Dutta

Positional Number Systems

Example: Using 10's complement, subtract 3250 - 72532.

$$\begin{aligned} M &= \quad 03250 \\ \text{10's complement of } N &= +\ 27468 \\ \text{Sum} &= \quad 30718 \end{aligned}$$

There is no end carry. Therefore, the answer is - (10's complement of 30718) = - 69282.

Note that since 3250 < 72532, the result is negative.

Because we are dealing with unsigned numbers, there is really no way to get an unsigned result for this case. When subtracting with complements, we recognize the negative answer from the absence of the end carry and the complemented result.

Dr. Arindam Dutta

Positional Number Systems

Example: Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$ and (b) $Y - X$ by using 2's complements.

(a)

$$
\begin{aligned}
X &= \quad 1010100 \\
\text{2's complement of } Y &= + \quad 0111101 \\
\text{Sum} &= \quad 10010001 \\
\text{Discard end carry } 2^7 &= -10000000 \\
\textit{Answer: } X - Y &= \quad 0010001
\end{aligned}
$$

(b)

$$
\begin{aligned}
Y &= \quad 1000011 \\
\text{2's complement of } X &= + \quad 0101100 \\
\text{Sum} &= \quad 1101111
\end{aligned}
$$

There is no end carry. Therefore, the answer is $Y - X = -$ (2's complement of 1101111) = - 0010001.

Dr. Arindam Dutta

Positional Number Systems

Example: Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$ and (b) $Y - X$ by using 1's complements.

(a) $X - Y = 1010100 - 1000011$

$X =$	1010100
1's complement of $Y =$	+ 0111100
Sum =	10010000
End-around carry = +	1
Answer: $X - Y =$	0010001

The (r - 1)'s complement is one less than the r's complement. Thus, the result of adding the minuend to the complement of the subtrahend produces a sum that is one less than the correct difference when an end carry occurs. Removing the end carry and adding 1 to the sum is referred to as an end-around carry.

(b) $Y - X = 1000011 - 1010100$

$Y =$	1000011
1's complement of $X =$	+ 0101011
Sum =	1101110

There is no end carry. Therefore, the answer is $Y - X = -$ (1's complement of 1101110) = - 0010001.

The negative result is obtained by taking the 1's complement of the sum. The procedure with end-around carry is also applicable to subtracting unsigned decimal numbers with 9's complement.

Dr. Arindam Dutta

Positional Number Systems

Signed Binary Numbers

- To represent negative integers, we need a notation for negative values.
- In ordinary arithmetic, negative number is indicated by – and a positive number by +.
- Computers represent the sign with a bit placed in the leftmost position of the number.
- The convention is to make the sign bit 0 for positive and 1 for negative.
- If the binary number is signed, the leftmost bit represents the sign and the rest of the bits represent the number. If it is assumed to be unsigned, then the leftmost bit is the most significant bit of the number.
- 01001 can be considered as 9 (unsigned binary) or as +9 (signed binary) because the leftmost bit is 0.
- 11001 represents the binary equivalent of 25 when considered as an unsigned number and the binary equivalent of -9 when considered as a signed number. This is because the leftmost bit is 1 (negative).

Dr Arindam Dutta

Positional Number Systems

Signed-magnitude convention: number consists of a magnitude and a symbol (+ or -) or a bit (0 or 1) indicating the sign.

Signed-complement system (1's or 2's): for representing negative numbers. Here, a negative number is indicated by its complement.

Example: Consider the number 9, represented in binary with eight bits. +9 is represented with a sign bit of 0 in the leftmost position, followed by the binary equivalent of 9, which gives 00001001. Although there is only one way to represent +9, there are three different ways to represent -9 with eight bits:

signed-magnitude representation: 10001001

signed-1's-complement representation: 11110110

signed-2's-complement representation: 11110111

In signed-magnitude, -9 is obtained from +9 by changing only the sign bit in the leftmost position from 0 to 1. In signed-1's-complement, -9 is obtained by complementing all the bits of +9, including the sign bit. The signed-2's-complement representation of -9 is obtained by taking the 2's complement of the positive number, including the sign bit.

Dr. Arindam Dutta

Positional Number Systems

All possible four-bit signed binary numbers in the three representations:

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
−0	—	1111	1000
−1	1111	1110	1001
−2	1110	1101	1010
−3	1101	1100	1011
−4	1100	1011	1100
−5	1011	1010	1101
−6	1010	1001	1110
−7	1001	1000	1111
−8	1000	—	—

Dr. Arindam Dutta

Positional Number Systems

Arithmetic Addition

- The addition of two numbers in the **signed-magnitude system** follows the rules of ordinary arithmetic. If the signs are the same, we add the two magnitudes and give the sum the common sign. If the signs are different, we subtract the smaller magnitude from the larger and give the difference the sign of the larger magnitude. For example, (+25) + (-37) = -(37 - 25) = -12 is done by subtracting the smaller magnitude, 25, from the larger magnitude, 37, and appending the sign of 37 to the result.

- For adding numbers in the **signed-complement system** does not require a comparison or subtraction, but only addition. The addition of two signed binary numbers with negative numbers represented in signed-2's-complement form is obtained from the addition of the two numbers, including their sign bits. A carry out of the sign-bit position is discarded.

+ 6	00000110	− 6	11111010
+13	00001101	+13	00001101
+19	00010011	+ 7	00000111
+ 6	00000110	− 6	11111010
−13	11110011	−13	11110011
− 7	11111001	−19	11101101

- If the sum obtained after the addition is negative, it is in 2's-complement form.

Dr Arindam Dutta

Positional Number Systems

Arithmetic Subtraction

- For subtraction of two signed binary numbers when negative numbers are in 2's-complement form, take the 2's complement of the subtrahend (including the sign bit) and add it to the minuend (including the sign bit). A carry out of the sign-bit position is discarded.

$$(\pm A) - (+B) = (\pm A) + (-B);$$
$$(\pm A) - (-B) = (\pm A) + (+B).$$

- Consider the subtraction (-6) - (-13) = +7. In binary with eight bits, this operation is written as (11111010 - 11110011). The subtraction is changed to addition by taking the 2's complement of the subtrahend (-13), giving (+13). In binary, this is 11111010 + 00001101 = 100000111. Removing the end carry, we obtain the correct answer: 00000111 (+7).

Dr. Arindam Dutta

ECE2002
DIGITAL LOGIC DESIGN

Module 1 – Lecture 2

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Binary Logic

- Binary logic consists of binary variables and a set of logical operations. The variables are designated by letters of the alphabet, such as A, B, C, x, y, z, etc., with each variable having two and only two distinct possible values: 1 and 0.

- There are three basic logical operations:

  1. **AND:** represented as $x \cdot y = z$ or $xy = z$ is read "x AND y is equal to z." The logical operation AND means $z = 1$ if and only if $x = 1$ and $y = 1$; otherwise, $z = 0$.

  2. **OR:** represented as $x + y = z$ is read "x OR y is equal to z," meaning that $z = 1$ if $x = 1$ or if $y = 1$ or if both $x = 1$ and $y = 1$. If both $x = 0$ and $y = 0$, then $z = 0$.

  3. **NOT:** represented as $x' = z$ (or $\bar{x} = z$) is read "not x is equal to z". In other words, if $x = 1$, then $z = 0$, but if $x = 0$, then $z = 1$. The NOT operation is also referred to as the *complement* operation.

- Binary logic resembles binary arithmetic, and the operations AND and OR have similarities to multiplication and addition, respectively.

- However, in binary arithmetic, we have $1 + 1 = 10$ (read "one plus one is equal to 2"), whereas in binary logic, we have $1 + 1 = 1$ (read "one OR one is equal to one").

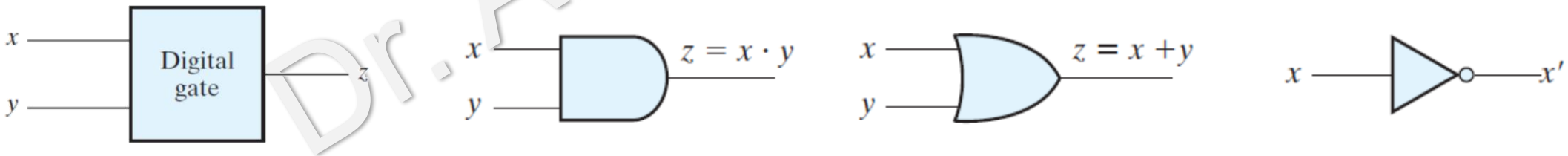
Dr. Arindam Dutta

Binary Logic

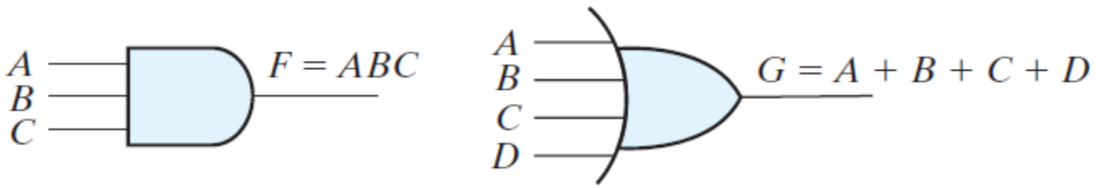
Truth table: A table of all possible combinations of the variables, showing the relation between the values that the variables may take and the result of the operation. For each combination, the result of the operation is then listed in a separate row.

AND			OR			NOT	
x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Logic gates: electronic circuits that operate on one or more input signals to produce an output signal.



AND and OR gates may have more than two inputs.



Dr. Arindam Dutta

Boolean Algebra

In 1854, George Boole developed an algebraic system now called Boolean algebra. It is an algebraic structure defined by a set of elements, B, together with two binary operators, + and ., provided that the following **Huntington postulates** are satisfied:

1. (a) The structure is *closed* with respect to the operator +.
(b) The structure is *closed* with respect to the operator ..

2. (a) The element 0 is an *identity* element with respect to +; that is, $x + 0 = 0 + x = x$.
(b) The element 1 is an *identity* element with respect to .; that is, $x \,.\, 1 = 1 \,.\, x = x$.

3. (a) The structure is *commutative* with respect to +; that is, $x + y = y + x$.
(b) The structure is *commutative* with respect to . ; that is, $x \,.\, y = y \,.\, x$.

4. (a) The operator . is *distributive* over +; that is, $x \,.\, (y + z) = (x \,.\, y) + (x \,.\, z)$.
(b) The operator + is *distributive* over .; that is, $x + (y \,.\, z) = (x + y) \,.\, (x + z)$.

5. For every element $x \in B$, there exists an element $x' \in B$ (called the *complement* of x) such that (a) $x + x' = 1$ and (b) $x \,.\, x' = 0$.

6. There exist at least two elements $x, y \in B$ such that $x \neq y$.

Dr. Arindam Dutta

Boolean Algebra

Postulates and Theorems of Boolean Algebra

Identity	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Inverse	(a)	$x + x' = 1$	(b)	$x \cdot x' = 0$
Idempotent	(a)	$x + x = x$	(b)	$x \cdot x = x$
Null or Dominance	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Double Complement		$(x')' = x$		
Commutative	(a)	$x + y = y + x$	(b)	$xy = yx$
Associative	(a)	$x + (y + z) = (x + y) + z$	(b)	$x(yz) = (xy)z$
Distributive	(a)	$x(y + z) = xy + xz$	(b)	$x + yz = (x + y)(x + z)$
De-Morgan's	(a)	$(x + y)' = x'y'$	(b)	$(xy)' = x' + y'$
Absorption	(a)	$x + xy = x$	(b)	$x(x + y) = x$

Dr. Arindam Dutta

Boolean Algebra

Redundancy theorem: In a Boolean expression, if the same factor is present in more than one term, the other terms are considered redundant, e.g. A + AB = A (here term A is present in both the terms, and hence the other term is neglected)

$$AB + ABC + A\bar{B} = AB + A\bar{B}$$
$$= A$$

Consensus theorem: If in a Boolean expression, one term contains a variable and the second term contains the complement of this variable, then in the expression a third term can be added without effecting the value of the expression, e.g. if X = AC+ BC' where

$$X = AC + B\bar{C} + AB$$

Duality principle: In the Huntington postulates, part (a)/(b) may be obtained from (b)/(a) if the binary operators and the identity elements are interchanged. This is called the duality principle.

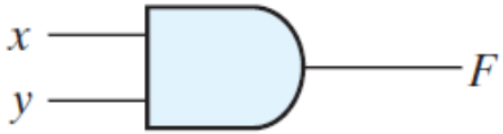
It states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. Given a logic expression, its *dual* is obtained by replacing all + operators with · operators, and vice versa, and by replacing all 0s with 1s, and vice versa.

Dr. Arindam Dutta

Digital Logic Gates

Basic Logic Gates

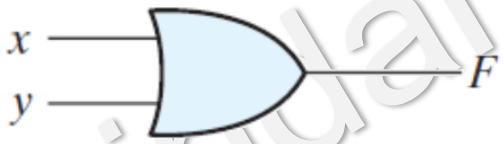
1. AND



$F = x \cdot y$

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

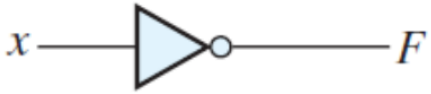
2. OR



$F = x + y$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

3. Inverter



$F = x'$

x	F
0	1
1	0

4. Buffer



$F = x$

x	F
0	0
1	1

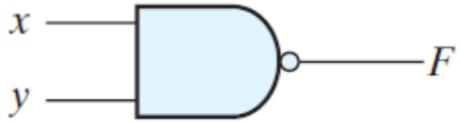
Dr. Arindam Dutta

Digital Logic Gates

Universal Logic Gates

5

NAND

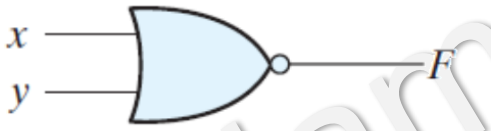


$F = (xy)'$

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

6

NOR

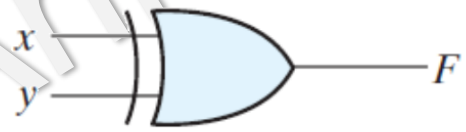


$F = (x + y)'$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

7

Exclusive-OR (XOR)

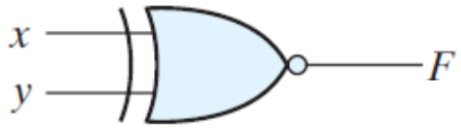


$F = xy' + x'y$
$= x \oplus y$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

8

Exclusive-NOR or equivalence



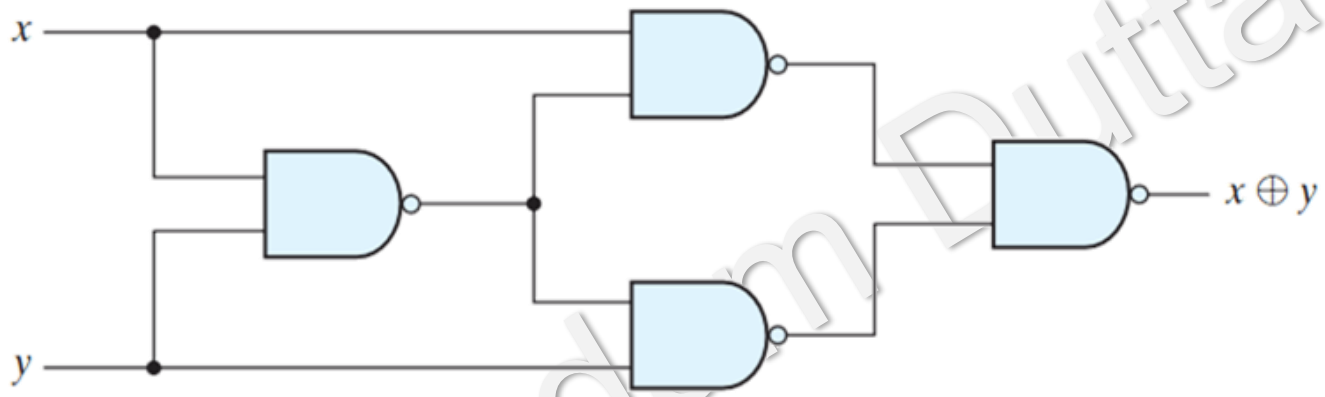
$F = xy + x'y'$
$= (x \oplus y)'$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

Dr. Arindam Dutta

Digital Logic Gates

Realization of EX-OR gate using only NAND gates



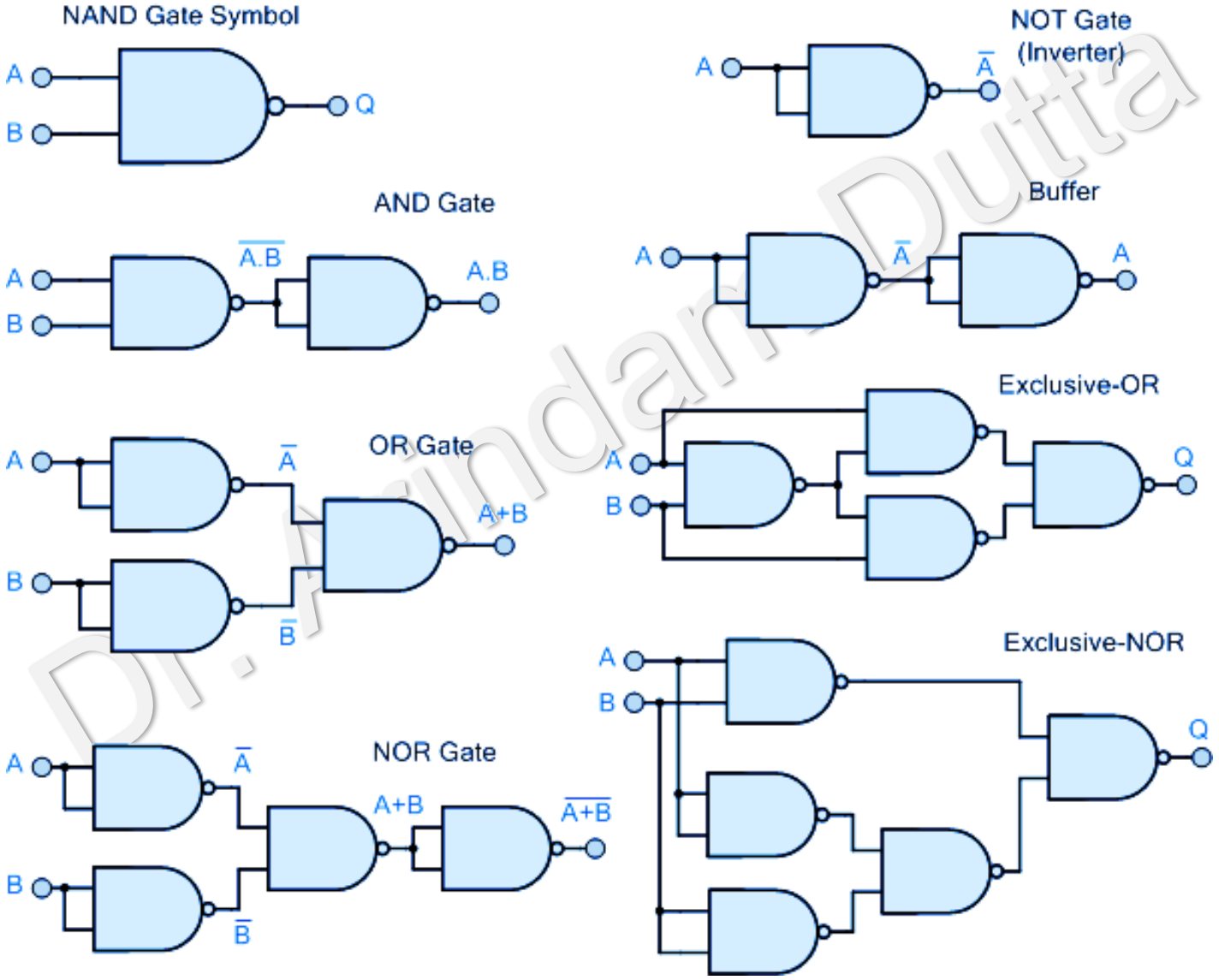
The first NAND gate performs the operation $(xy)' = (x' + y')$. The other two-level NAND circuit produces the sum of products of its inputs:

$$(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$$

Dr. Arindam Dutta

Digital Logic Gates

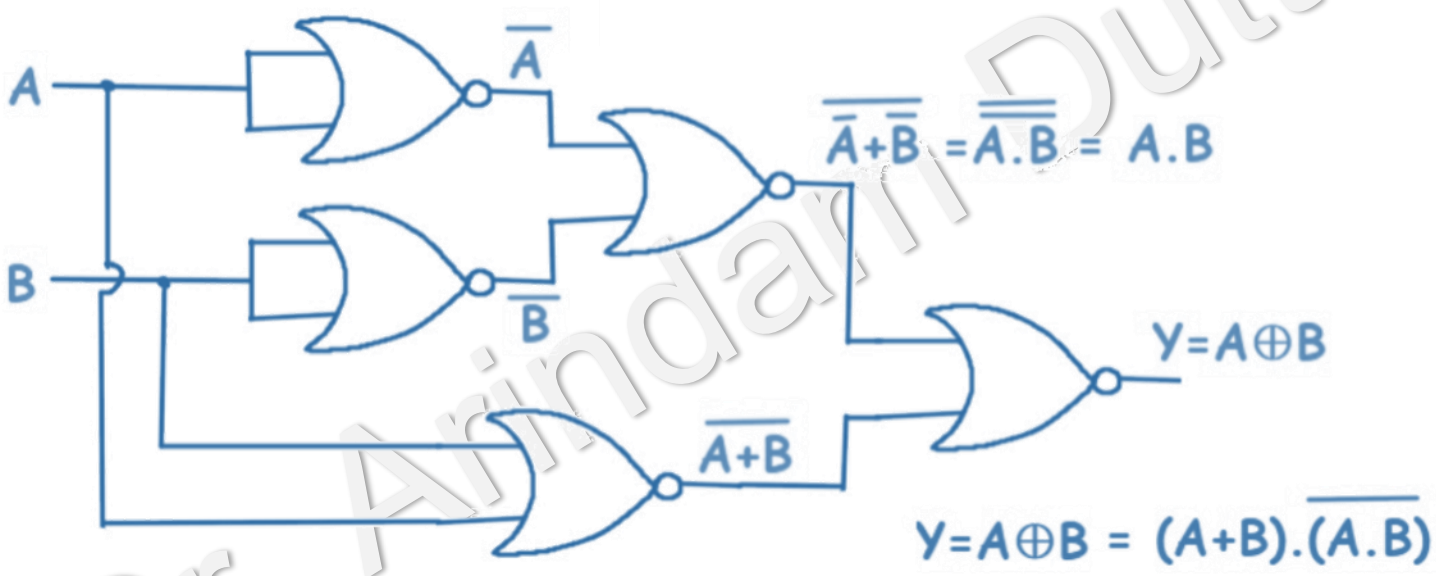
NAND as universal gate



Dr. Arindam Dutta

Digital Logic Gates

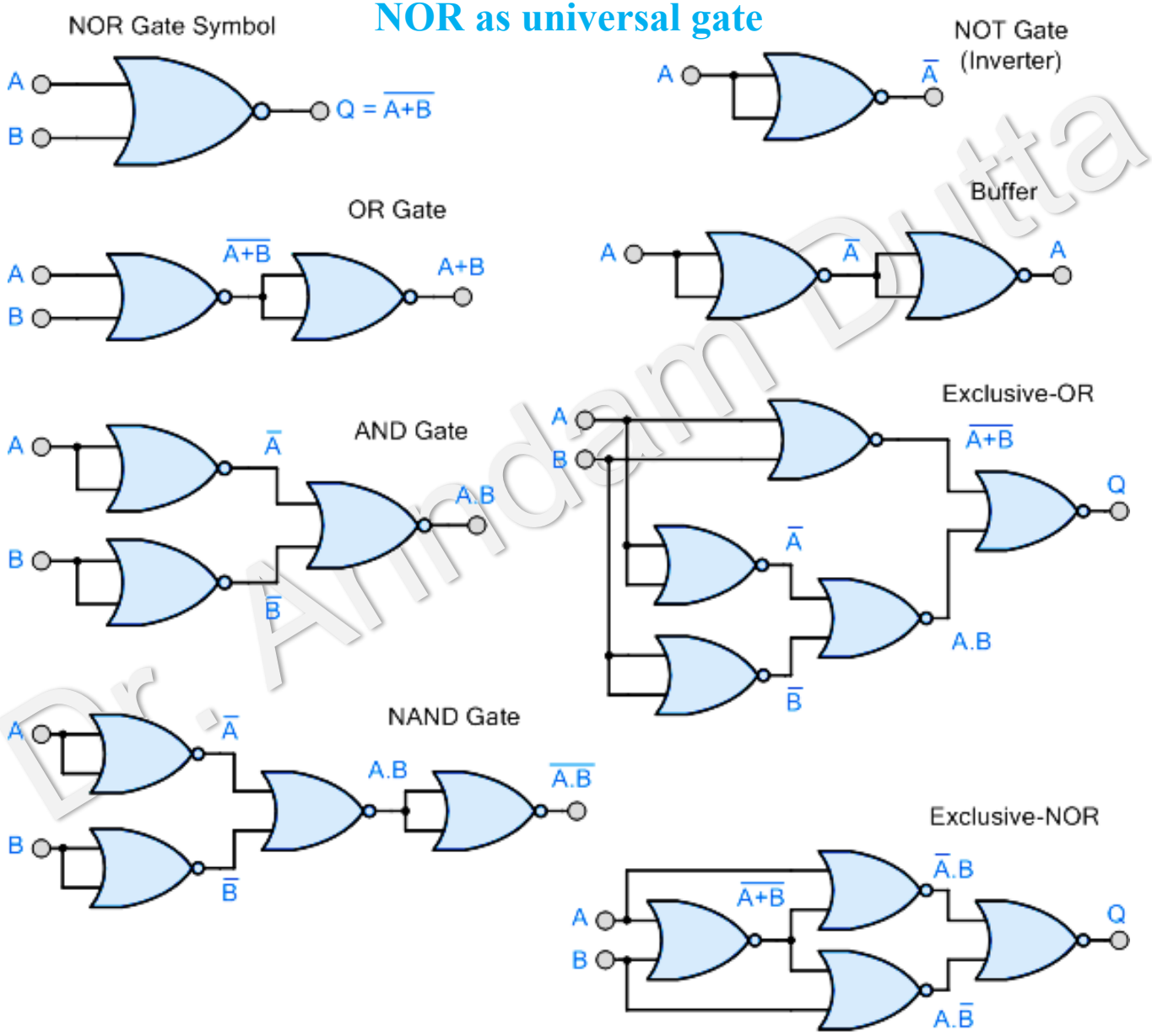
Realization of EX-OR gate using only NOR gates



Dr. Arindam Dutta

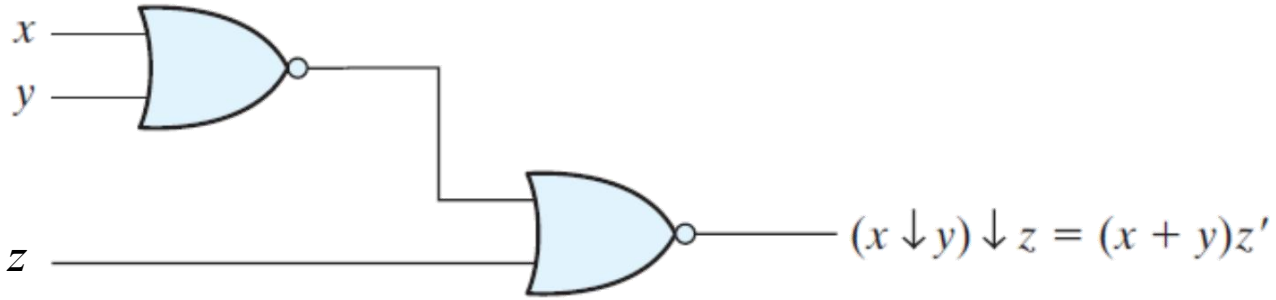
Digital Logic Gates

NOR as universal gate

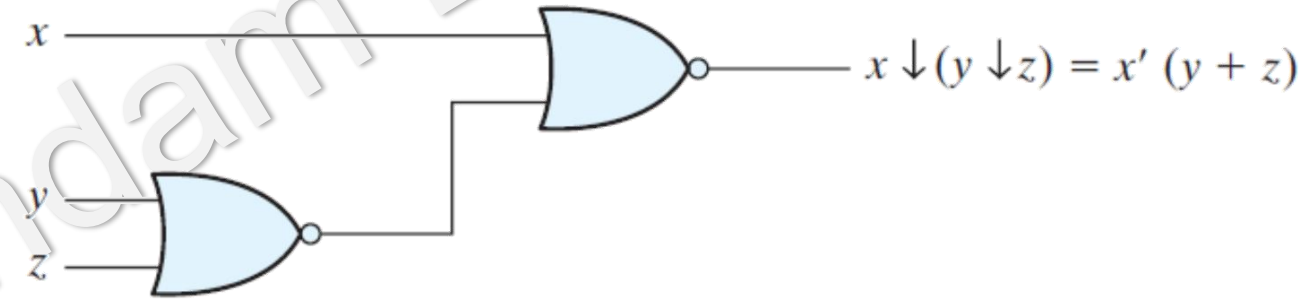


Dr. Arindam Dutta

Digital Logic Gates

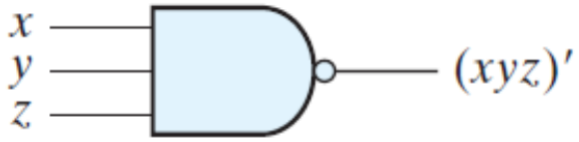


(↓ indicates NOR operation)





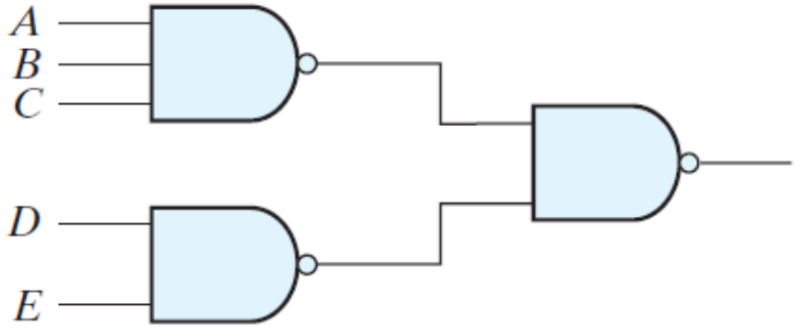
3-input NOR gate



3-input NAND gate

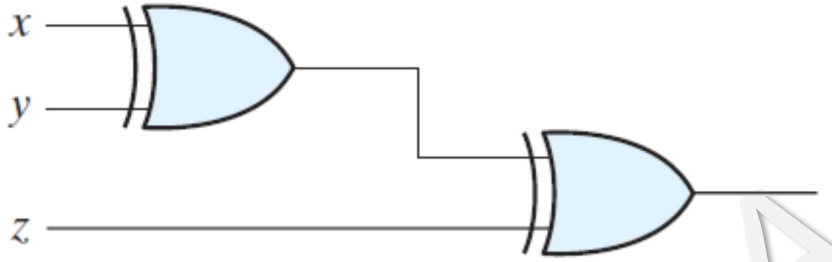
Dr. Arindam Dutta

Digital Logic Gates



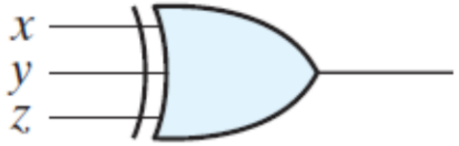
$$F = [(ABC)' \cdot (DE)']' = ABC + DE$$

Cascaded NAND gates



$$F = x \oplus y \oplus z$$

2-input XOR gates



$$F = x \oplus y \oplus z$$

3-input XOR gate

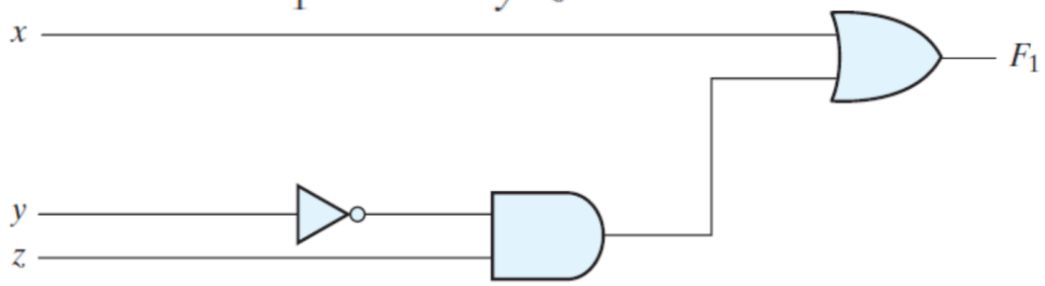
x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Dr. Arindam Dutta

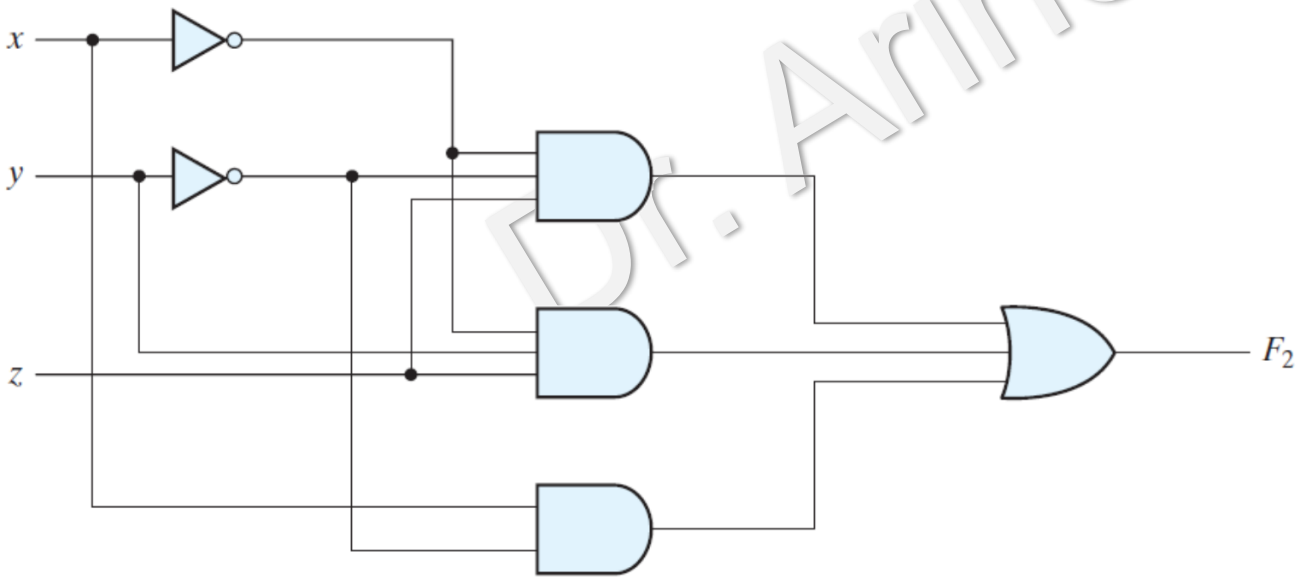
Boolean Functions

A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols. For a given value of the binary variables, the function can be equal to either 1 or 0. Example:

$$F_1 = x + y'z$$



$$F_2 = x'y'z + x'yz + xy'$$



x	y	z	F_1	F_2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0

Dr. Arindam Dutta

Boolean Functions

Using the postulates and theorems of Boolean algebra, F_2 can be simplified as:

$$F_2 = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$$

The function F_2 initially had three terms and eight literals, now it has two terms and four literals. By reducing the number of terms, the number of literals, or both in a Boolean expression, it is often possible to obtain a simpler circuit.



Examples: **1.** $x(x' + y) = xx' + xy = 0 + xy = xy.$

2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y.$

3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x.$

4.
$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z. \end{aligned}$$

5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$, by duality from function 4.

Dr. Arindam Dutta

Boolean Functions

The **complement of a function** F is F' and is obtained from an interchange of 0's for 1's and 1's for 0's in the value of F. The complement of a function may be derived algebraically through DeMorgan's theorems.

Example: Find the complement of the functions $F_1 = x'y z' + x'y'z$ and $F_2 = x\,(y'z' + y z)$.

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F_2' = [x(y'z' + yz)]' &= x' + (y'z' + yz)' = x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') \\ &= x' + yz' + y'z \end{aligned}$$

A simpler procedure for deriving the complement of a function is to take the dual of the function and complement each literal.

1. $F_1 = x'yz' + x'y'z$.
   The dual of F_1 is $(x' + y + z')(x' + y' + z)$.
   Complement each literal: $(x + y' + z)(x + y + z') = F_1'$.
2. $F_2 = x(y'z' + yz)$.
   The dual of F_2 is $x + (y' + z')(y + z)$.
   Complement each literal: $x' + (y + z)(y' + z') = F_2'$.

Dr. Arindam Dutta

Canonical and Standard Forms

Minterms and Maxterms: When two binary variables x and y are combined with an AND operation, there are four possible combinations: $x'y'$, $x'y$, xy', and xy. Each of these four AND terms is called a **minterm**, or a **standard product**. In a similar manner, n variables can be combined to form 2^n minterms.

In a similar fashion, n variables forming an OR term, with each variable being primed or unprimed, provide 2^n possible combinations, called **maxterms**, or **standard sums**.

			Minterms		Maxterms	
x	y	z	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Dr. Arindam Dutta

Canonical and Standard Forms

A Boolean function can be expressed algebraically from a given truth table by forming a minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.

The function f_1 is determined by expressing the combinations 001, 100, and 111 as $x'y'z$, $xy'z'$, and xyz, respectively. Since each one of these minterms results in $f_1 = 1$, we have

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

Thus, any Boolean function can be expressed as a sum of minterms.

Now, the complement of f_1 from the truth table is:

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

If we take the complement of f_1', we obtain

$$\begin{aligned} f_1 &= (x + y + z)(x + y' + z)(x' + y + z')(x' + y' + z) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \end{aligned}$$

Thus, any Boolean function can be expressed as a product of maxterms.

Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form.

Dr. Arindam Dutta

Canonical and Standard Forms

Sum of minterms

The minterms whose sum defines the Boolean function are those which give the 1's of the function in a truth table.

Example: Express the Boolean function ***F = A + B'C*** as a sum of minterms.

After obtaining the truth table, we can read the five minterms of the function to be 1, 4, 5, 6, and 7.

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

Thus,
$$F = A'B'C + AB'C + AB'C + ABC' + ABC$$
$$= m_1 + m_4 + m_5 + m_6 + m_7$$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Dr. Arindam Dutta

Canonical and Standard Forms

Product of maxterms: Each of the 2^{2^n} functions of n binary variables can be also expressed as a product of maxterms. To express a Boolean function as a product of maxterms, it must first be brought into a form of OR terms. This may be done by using the distributive law, $x + yz = (x + y)(x + z)$. Then any missing variable x in each OR term is ORed with xx'. The procedure is clarified in the following example.

Example: Express the Boolean function $F = xy + x'z$ as a product of maxterms. First, convert the function into OR terms by using the distributive law:

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) \\ &= (x + x')(y + x')(x + z)(y + z) \\ &= (x' + y)(x + z)(y + z) \end{aligned}$$

The function has three variables: x, y, and z. Each OR term is missing one variable; therefore,

$$\begin{aligned} x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \end{aligned}$$

Combining all the terms and removing those which appear more than once, we finally obtain

$$\begin{aligned} F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\ &= M_0M_2M_4M_5 \end{aligned} \longrightarrow F(x, y, z) = \Pi(0, 2, 4, 5)$$

Dr Arindam Dutta

Canonical and Standard Forms

Conversion between canonical forms: Maxterm with subscript *j* is a complement of the minterm with the same subscript *j* and vice versa. That is, $m_j' = M_j$.

A Boolean function can be converted from an algebraic expression to a product of maxterms by means of a truth table and the canonical conversion procedure. For example:

$$F = xy + x'z$$

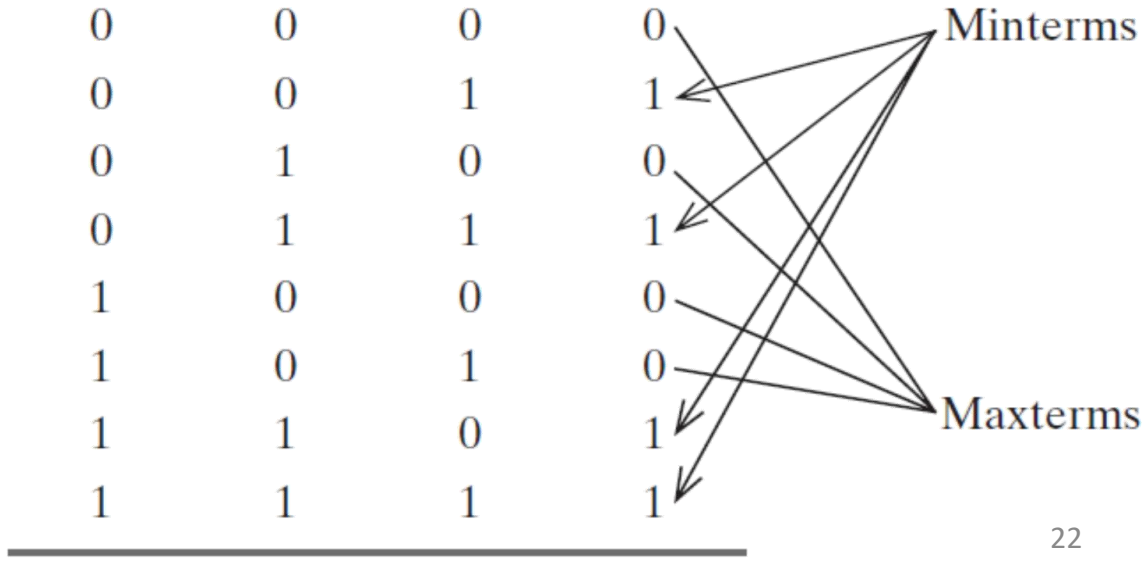
The truth table is derived and the minterms of the function are read from the truth table to be 1, 3, 6, and 7. The function expressed as a sum of minterms is:

$$F(x, y, z) = \Sigma(1, 3, 6, 7)$$

Since there is a total of eight minterms or maxterms in a function of three variables, we determine the missing terms to be 0, 2, 4, and 5. The function expressed as a product of maxterms is

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



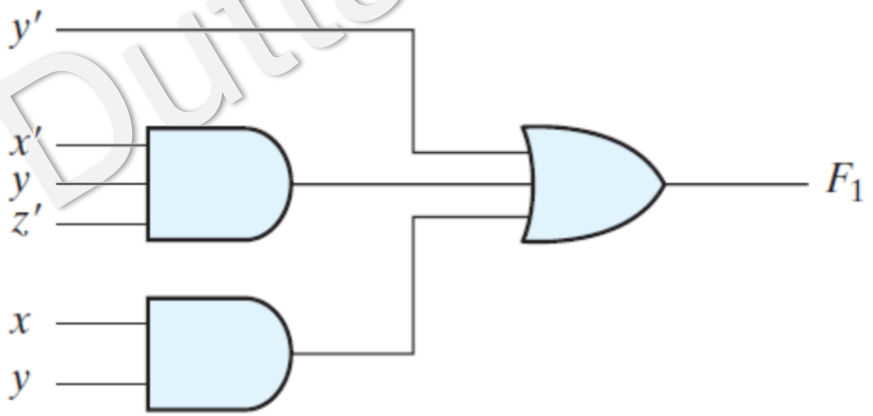
Dr. Arindam Dutta

Canonical and Standard Forms

Standard form: Terms that form the function may contain one, two, or any number of literals. There are two types of standard forms:

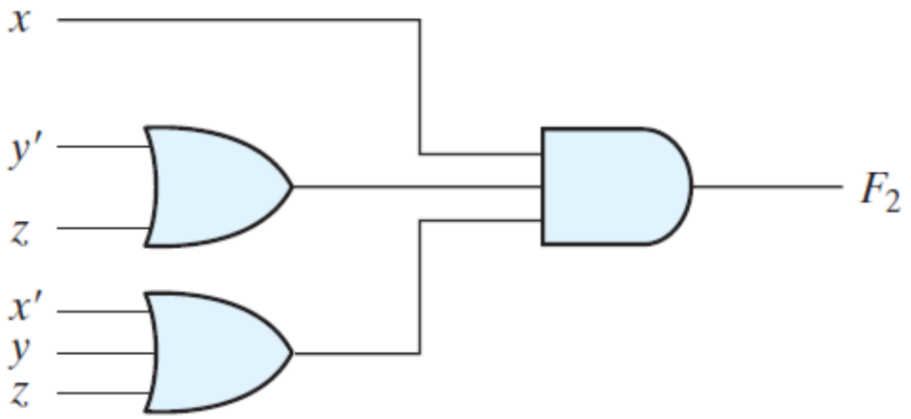
Sum of products (SOP): It is a Boolean expression containing AND terms, called *product terms,* with one or more literals each. The *sum* denotes the ORing of these terms. Example:

$$F_1 = y' + xy + x'yz'$$



Products of sums (POS): It is a Boolean expression containing OR terms, called *sum terms*. Each term may have any number of literals. The *product* denotes the ANDing of these terms. Example:

$$F_2 = x(y' + z)(x' + y + z')$$

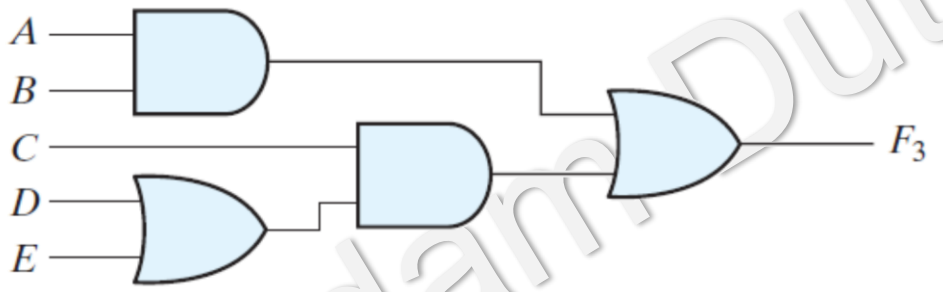


Dr. Arindam Dutta

Canonical and Standard Forms

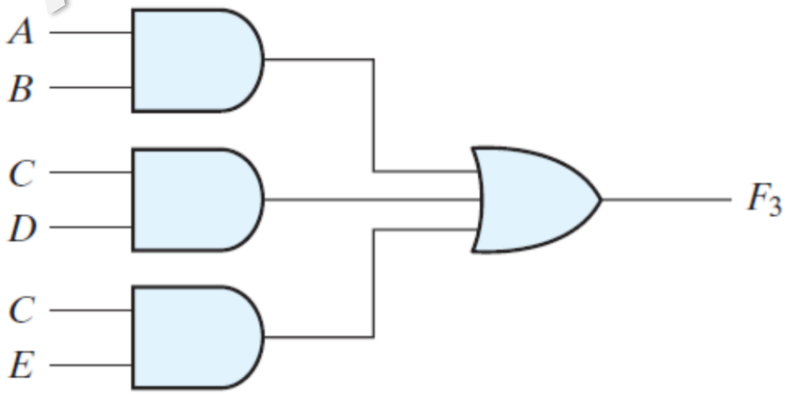
A Boolean function may be expressed in a nonstandard form. For example, the function

$$F_3 = AB + C\,(D + E)$$



It can be changed to a standard form by using the distributive law to remove the parentheses:

$$F_3 = AB + C(D + E) = AB + CD + CE$$



Dr. Arindam Dutta

ECE2002
DIGITAL LOGIC DESIGN

Module 1 – Lecture 3

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Digital Logic Families

- Digital integrated circuits (ICs) are classified not only by their complexity or logical operation, but also by the specific circuit technology to which they belong. The circuit technology is referred to as a *digital logic family*.

- Each logic family has its own basic electronic circuit upon which more complex digital circuits and components are developed.

- The basic circuit in each technology is typically a NAND, NOR, and/or NOT (inverter) gate.

- The electronic components employed in the construction of the basic circuit are usually used to name the technology.

- The following are some of the most popular logic families of digital integrated circuits :

  - **Resistor-transistor logic (RTL):** suitable for slow applications
  - **Diode-transistor logic (DTL):** commonly used because of its economical aspect
  - **Transistor-transistor logic (TTL):** has been in use for 50 years and is considered to be standard
  - **Emitter-coupled logic (ECL):** has an advantage in systems requiring high-speed operation
  - **Current-mode logic (CML):** suitable for high-frequency applications
  - **Metal-oxide semiconductor (MOS):** suitable for circuits that need high component density
  - **Complementary metal-oxide semiconductor (CMOS):** preferable in systems requiring low power consumption, such as digital cameras, personal media players, and other handheld portable devices.

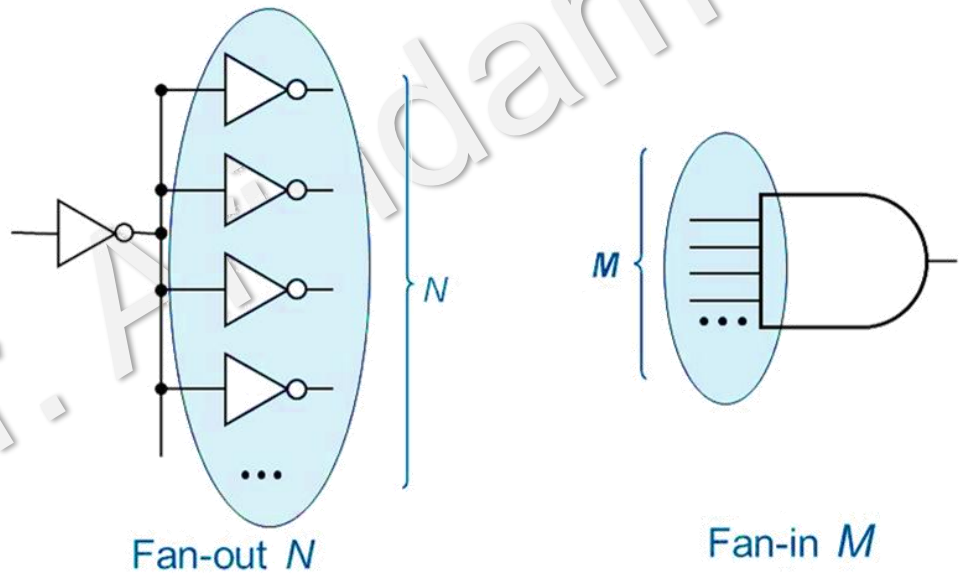
Dr Arindam Dutta

Digital Logic Families

The most important **parameters distinguishing logic families** are listed below:

- **Fan-out** specifies the number of standard loads that the output of a typical gate can drive without impairing its normal operation. A standard load is usually defined as the amount of current needed by an input of another similar gate in the same family.

- **Fan-in** is the number of inputs available in a gate. For instance, in an AND gate, the fan-in is 2, whereas in a NOT gate, it is 1.



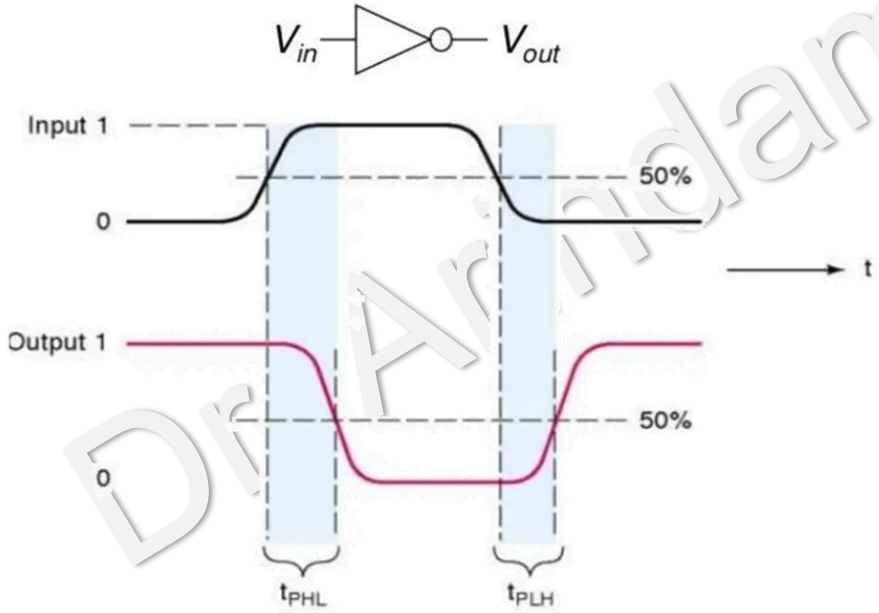
- **Noise margin** **or** **noise immunity** is the maximum external noise voltage added to an input signal that does not cause an undesirable change in the circuit output.

Dr. Arindam Dutta

Digital Logic Families

- **Power dissipation** is the power consumed by the gate that must be available from the power supply.

- **Propagation delay** is the average transition delay time for a signal to propagate from input to output. For example, if the input of an inverter switches from 0 to 1, the output will switch from 1 to 0, but after a time determined by the propagation delay of the device. It is the average of the time taken by the output to go from low to high (t_{PLH}) and for it to go from high to low (t_{PHL}).



$$t_P = (t_{PLH} + t_{PHL}) / 2$$

The propagation delay time is inversely proportional to the operating speed and directly proportional to the switching time and increases as the fan-out increases.

Dr Arindam Dutta

Digital Logic Families

RTL (resistor-transistor logic):

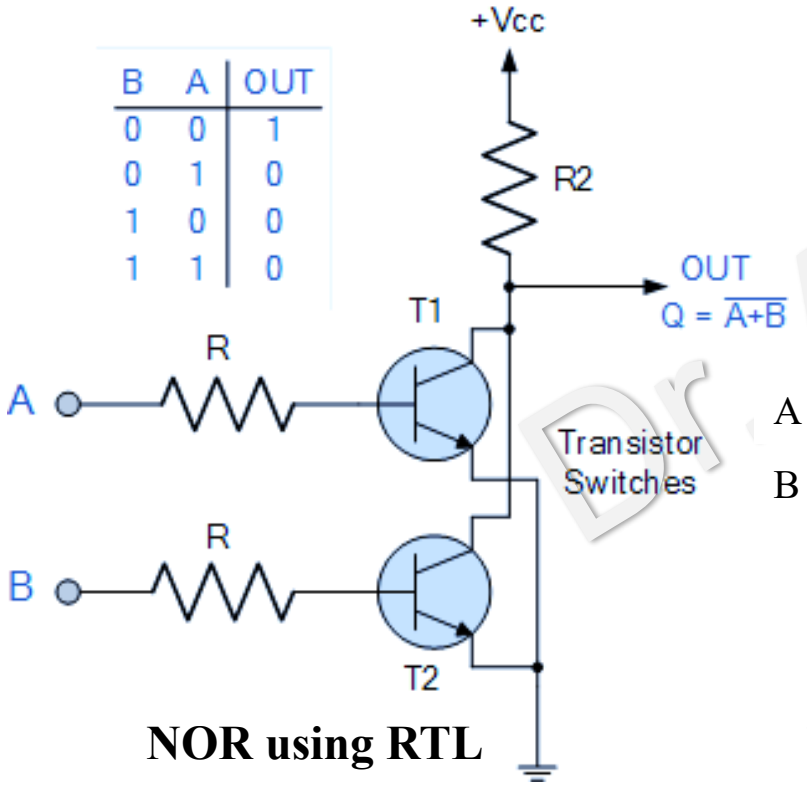
- It is a bipolar transistor logic family.
- The basic circuit of the RTL logic family is the NOR.
- Each input is associated with one resistor and one transistor.
- **Working:**
  - **If any input is high**, the corresponding transistor is driven into saturation and the output goes low, regardless of the states of the other transistor.
  - **If all inputs are low**, then all transistors are in cutoff state and the output of the circuit goes high.
- **Characteristics:**
  - It has a fan-out of 5
  - Propagation delay is 12 - 25 ns.
  - Power dissipation is 12 mW.
  - Noise margin for low signal input is 0.4 V.
  - Poor noise immunity.
  - Low speed.

Dr Arindam Dutta

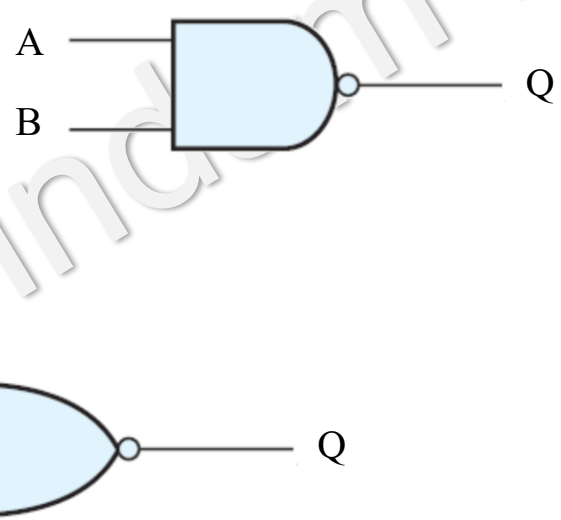
Digital Logic Families

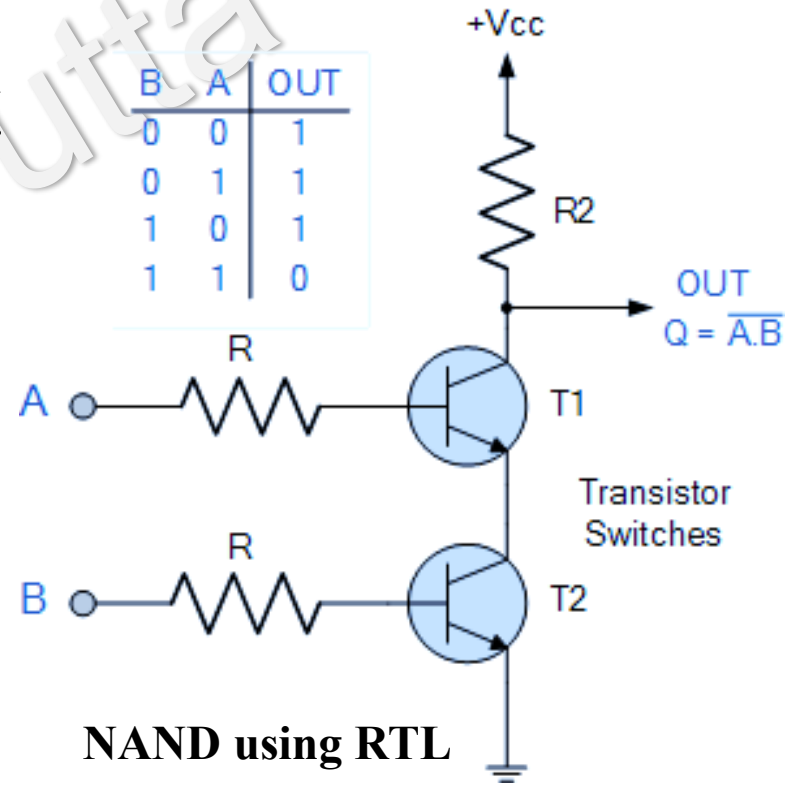
RTL (resistor-transistor logic): All the logic are implemented using resistors and transistors. One basic thing about the transistor (NPN), is that HIGH at input causes output to be LOW (i.e. like an inverter).

A 2-input logic NAND gate can be constructed using RTL switches connected together with the inputs connected directly to the transistor bases. Either transistor must be cut-off or "OFF" for an output at Q.



NOR using RTL





NAND using RTL

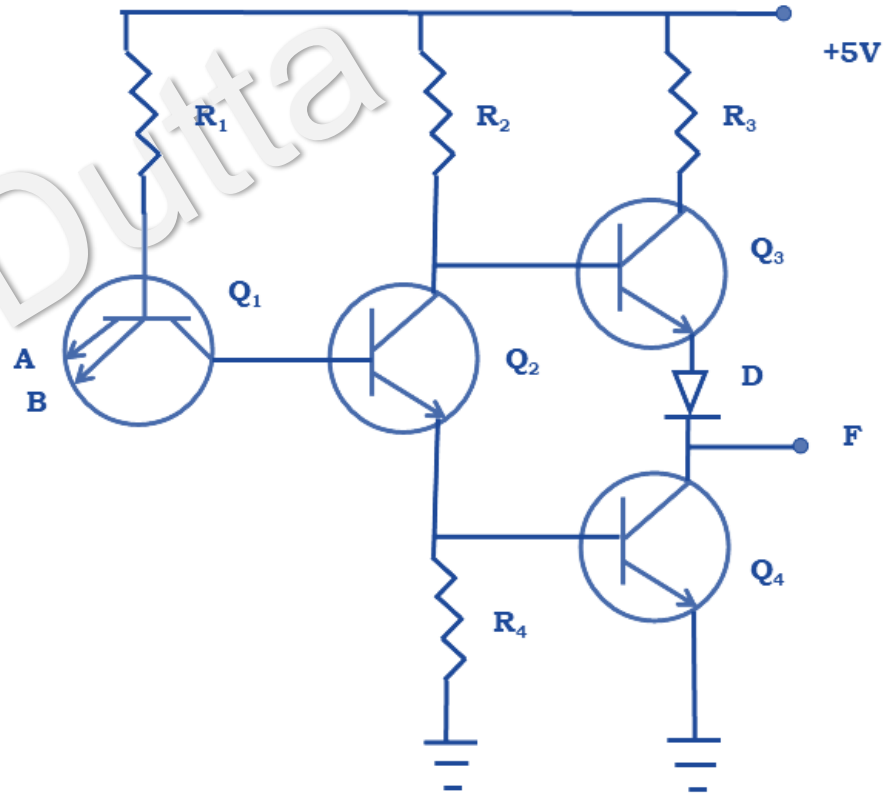
A 2-input NOR gate can be constructed using RTL switches connected together with the inputs connected directly to the transistor bases. Both transistors must be cut-off or "OFF" for an output at Q.

Dr. Arindam Dutta

Digital Logic Families

TTL (transistor-transistor logic):

- It is a bipolar logic family that can perform many digital functions and has achieved popularity.
- TTL ICs are given the numerical designation as 5400 and 7400 series.
- The basic circuit of TTL with totem pole output stage is NAND gate.
- TTL uses a multi-emitter transistor at the input and is fast saturation logic circuit.
- The output transistor Q3 and Q4 form a totem pole connection.
- This extra output stage is known as totem-pole stage because three output components Q3 and Q4 and Diode are stacked on one another.



- This arrangement will increase the speed of operation and also increase output current capability.
- The function of the diode in this circuit is to prevent both Q3 and Q4 from being turned ON simultaneously.

Dr. Arindam Dutta

Digital Logic Families

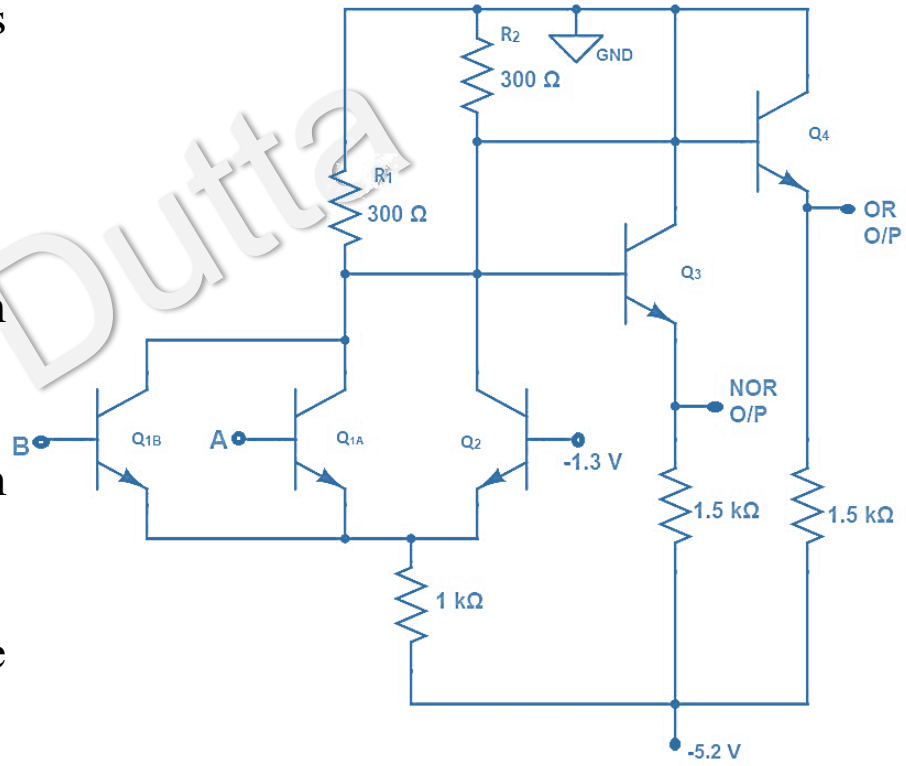
Characteristics of TTL:

- TTL has greater speed than RTL and DTL.
- Less to moderate noise immunity. TTL will take anything below about 0.8 V as low (0), and anything above about 2 V as a high (1).
- Power dissipation is10 mW.
- It has fan-in of 6 and fan-out of 10.
- Propagation time delay is 5-15 ns.

Dr. Arindam Dutta

Digital Logic Families

ECL (emitter-coupled logic): It is a bipolar transistor logic family that is considered to be one of the fastest logics available.



- It was invented in 1956 at IBM by Hannon S. Yourke.
- ECL is also known as current-steering logic (CSL) or current-switch emitter-follower (CSEF) logic.
- The transistors are not allowed to be operated in the saturation region thereby improving the speed of circuit to a great extent.
- Based on BJT, but removes problems of delay time by preventing the transistors from saturating.
- The output of ECL provides OR and NOR functions.
- ECL is used in high-performance applications, such as: Clock-distribution circuits, High-frequency-based applications, Fiber-optic transceiver interfaces, Ethernet, and ATM (Asynchronous Transfer Mode) networks.

Dr. Arindam Dutta

Digital Logic Families

Characteristics of ECL:

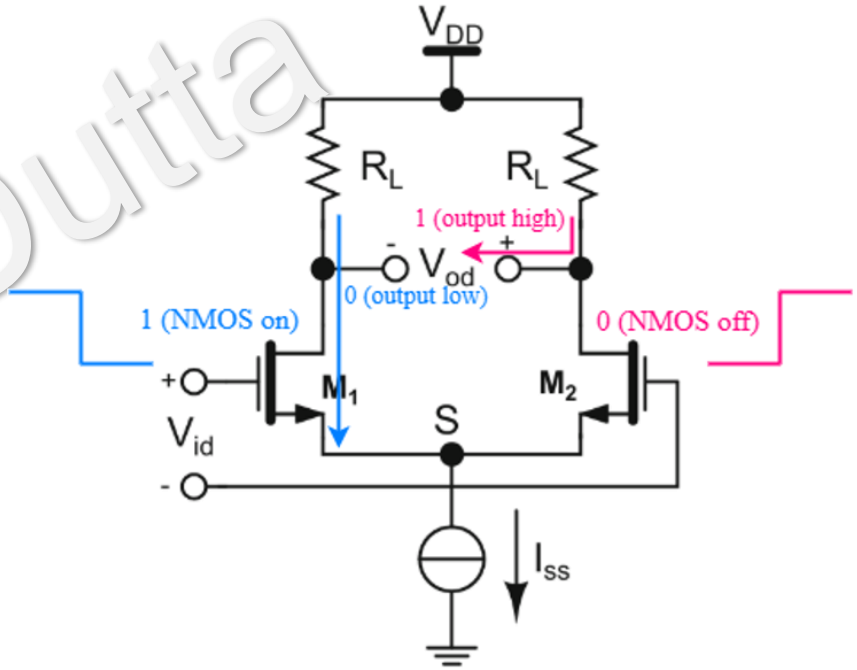
- Very fast operation; propagation delays of 1ns or less.
- Low noise immunity of about 0.2 - 0.25 V.
- ECL circuit usually operates with –ve supply (+ve terminal is connected to ground). In this logic family, we consider the logic 0 as -1.6 V and logic 1 as -0.8 V.
- The input impedance is high and the output impedance is low. As a result, the transistors change states quickly, gate delays are low, and the fan out capability is high.

Dr. Arindam Dutta

Digital Logic Families

CML (Current-mode logic):

- It is a high-speed logic family that performs logic operations by steering a constant current through differential transistor pairs.
- Unlike ECL, which is traditionally implemented using BJTs, modern CML can also be implemented with MOSFETs.
- Uses small differential voltage swings (typically 400–800 mV), enabling very high-speed switching with low signal distortion.
- Consumes nearly constant power because a constant current source remains active during operation.



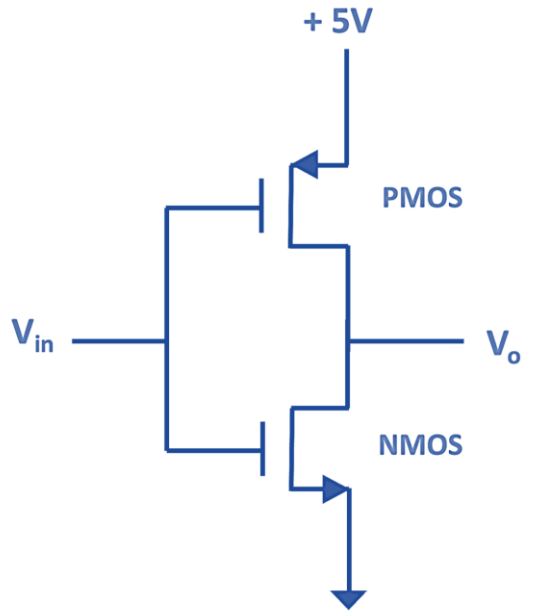
- Provides excellent signal integrity, low jitter, and high immunity to common-mode noise due to differential signalling.
- Widely used in multi-Gbps serial interfaces, clock distribution networks, optical communication systems, and high-speed I/O circuits.

Dr. Arindam Dutta

Digital Logic Families

CMOS (Complementary metal-oxide semiconductor):

- It is a unipolar logic family with NOT as the fundamental gate.
- Both n-type and p-type transistors are used to design logical functions. The same signal which turns ON one transistor is used to turn OFF the other transistor.
- Considerably lower energy consumption than TTL and ECL, which has made portable electronics possible. Low power consumption is essential for VLSI design; therefore, CMOS has become the dominant logic family, while TTL and ECL continue to decline in use.
- Most widely used family for large-scale devices.
- Combines high speed with low power consumption.
- Usually operates from a single supply of 5 – 15 V.
- Excellent noise immunity of about 30% of supply voltage.
- Can be connected to a large number of gates (about 50).



Dr. Arindam Dutta

Digital Logic Families

Parameter	RTL	TTL	ECL	CML	CMOS
Basic Gate	NOR	NAND	OR/NOR	NOR/NAND	NOT
Basic Device	Bipolar transistor + resistors	Bipolar transistors	Bipolar differential pair	Bipolar/MOSFETs differential pair with constant current source	Complementary nMOS & pMOS FETs
Transistor Region	Saturation & cut-off	Saturation & cut-off	Active	Active	Cut-off & linear
Typical Supply Voltage	+3 to +5 V	+5 V	−5.2 V (typically)	+3.3 V or lower (technology dependent)	1–15 V (typically 5 V, 3.3 V, 1.8 V, etc.)
Propagation Delay	20–50 ns	3–15 ns	0.5–1 ns	0.05–1 ns	2–20 ns (technology dependent)
Power Dissipation	High	Moderate	Very High	High (constant current)	Very Low (static), moderate dynamic
Power-Delay Product	Poor	Moderate	Moderate	Good	Excellent
Noise Immunity	Poor	Moderate	Low to Moderate	Low	Excellent
Fan-out	5–10	10–20	20–30	High	Very High (50+)
Fan-in	Limited	Moderate	Moderate	Moderate	High
Operating Speed	Low	High	Very High	Extremely High	High to Very High
Switching Method	Saturation	Saturation	Non-saturation	Non-saturation	Charging/discharging of load capacitances

Dr. Arindam Dutta

Digital Logic Families

Parameter	RTL	TTL	ECL	CML	CMOS
Static Power Consumption	High	Moderate	Very High	High	Nearly Zero
Dynamic Power Consumption	Moderate	Moderate	High	High	Proportional to switching frequency
Integration Density	Low	Medium	Low	Medium	Very High (VLSI/ULSI)
Noise Margin	Small	Moderate	Small	Small	Large
Voltage Swing	Large	Moderate	Small (~0.8 V)	Small (typically 0.4–0.8 V)	Rail-to-rail
Input Impedance	Low	Moderate	High	High	Extremely High
Output Impedance	Moderate	Low	Low	Low	Low
Speed-Power Trade-off	Poor	Good	Favours speed	Optimized for very high speed	Favors low power
Fabrication Complexity	Very Simple	Moderate	Complex	Complex	Moderate to High
Typical Applications	Early computers, obsolete systems	Older microprocessors, industrial control	Supercomputers, high-speed communication	Multi-Gbps communication, SERDES, clock/data recovery	Modern processors, memories, ASICs, FPGAs, mobile devices
Major Advantage	Simple design	Good balance of speed and robustness	Fastest bipolar logic	Extremely high-speed operation	Lowest power and highest integration density
Major Disadvantage	Slow, high power, poor noise immunity	Higher power than CMOS	Very high power consumption	Constant power dissipation	Dynamic power increases with frequency

Dr. Arindam Dutta

Digital Logic Families

Logic Family	Greatest Advantage	Greatest Disadvantage
RTL	Simplest design and low implementation complexity	Slow operation, high power consumption, and poor noise immunity
TTL	Good balance of speed, power, and reliability	Higher power consumption than CMOS
ECL	Extremely high switching speed (fastest bipolar logic family)	Very high power consumption and complex power supply requirements
CML	Ultra-high-speed operation with small voltage swing and low jitter	Constant power dissipation and higher circuit complexity
CMOS	Lowest power consumption, high noise immunity, and very high integration density	Dynamic power consumption increases with switching frequency and load capacitance

Dr. Arindam Dutta

Digital Logic Families

Parameter	Best → Worst
Operating Speed	CML ≈ ECL > TTL > CMOS > RTL
Propagation Delay *(Lowest → Highest)*	CML ≈ ECL < TTL < CMOS < RTL
Power Consumption *(Lowest → Highest)*	CMOS < TTL < RTL < CML < ECL
Power Efficiency *(Highest → Lowest)*	CMOS > TTL > CML > ECL > RTL
Noise Immunity	CMOS > TTL > ECL ≈ CML > RTL
Integration Density	CMOS > CML > TTL > ECL > RTL
Fan-out	CMOS > ECL > CML > TTL > RTL
Fan-in	CMOS > TTL ≈ ECL ≈ CML > RTL
Cost (Modern Implementation) *(Lowest → Highest)*	CMOS < TTL < CML < ECL *(RTL is obsolete and generally not considered in modern implementations.)*

Note: *The rankings are generalized comparisons based on the most common implementations of these logic families. Actual values may vary depending on the specific technology node, IC series, and manufacturer.*

Dr. Arindam Dutta